

WorldEpisode: Diagnosing and Correcting Hidden World–Episode Failures in Robot-Learning Datasets

Alba Tellez Fernandez
URDF Studio
WorldEpisode draft maintainer

WorldEpisode Contributors

Abstract

Robot-learning datasets increasingly combine recorded actions, multimodal observations, reconstructed scenes, and simulator exports, yet the semantic relationship between an episode and the world in which it occurred is usually implicit. This creates hidden methodological failures: trajectory replay drifts when command timing and frame semantics are underspecified, and evaluation splits leak when episodes share reconstructed worlds, physical entities, or asset lineage. We introduce *WorldEpisode*, a formal, storage-neutral semantic contract that binds robot-learning episodes to immutable world revisions through five coupled graphs: identity, frame/clock, representation-role, temporal event/state, and provenance/derivation. The contract is not a replacement for LeRobotDataset, Rerun, NCore, MCAP, OpenUSD, or glTF Gaussian splats; it specifies the invariants those systems must preserve to support reproducible replay, conversion, and evaluation. We implement executable round-trip binding artifacts, an active public LeRobotDataset-to-WorldEpisode-to-LeRobotDataset conversion, a semantic validator, an independent fixture set, and controlled experiments. Across seven bindings, native containers preserve only 17–39% of the semantic contract, while WorldEpisode sidecars recover the complete contract for dataset/log/world bindings. On a pinned public SO-101 LeRobotDataset v3 episode, the active converter preserves 303 six-dimensional action rows, state rows, sample timestamps, two video timestamp ranges, and derived physical-frame records with zero measured numerical loss while explicitly reporting source-absent camera extrinsics, action units, and latency semantics. The validator detects all 14 injected fault classes and all hand-authored independent fixture failures. A real LeRobot trajectory reveals a four-frame, 133 ms effective action delay; timestamp-aware alignment reduces validation joint RMSE from 4.732 to 1.862 degrees, and tested MuJoCo replay drops from 3.425 to 1.563 degrees while the Isaac adapter remains ready but untested. On a public ArmnetBench LeRobot release, random episode splitting leaks all tested world lineages and gives a Torch behavioral-cloning baseline 0.850 offline imitation success, while a scene-disjoint split has zero lineage leakage and drops to 0.000. These results show that world–episode semantics are not documentation overhead; they are measurable controls for robot-learning validity.

1 Introduction

Robot-learning evaluation increasingly depends on a chain of conversions: recorded episodes become training datasets, reconstructed scenes become simulator assets, simulator rollouts become benchmark results, and generated counterfactuals become data augmentation. The chain is only as scientific as its hidden alignment assumptions. A camera transform with the wrong direction, an action stream interpreted in the wrong frame, a command timestamp treated as an actuation timestamp, or a train/test split that shares the same reconstructed world can change a conclusion without changing a dataset’s apparent file validity.

Existing formats solve important pieces of this chain. The LeRobot library vertically integrates robot learning from motor interfaces through dataset collection, storage, streaming, policies, and deployment [3]. The current LeRobotDataset v3 layout stores high-frequency states, actions, and timestamps in Parquet, visual streams in MP4 shards, and episode indexing in metadata rather than filenames [4]. Rerun provides a physical-data layer and storage target for multimodal recordings. NCore documents explicit pose graphs and sensor conventions. OpenUSD, SimReady, glTF, and simulator-native formats describe composed worlds and assets. Gaussian splats are also moving into standard asset containers through glTF’s `KHR_gaussian_splatting` extension [9, 10]. Recent real-to-sim systems such as RoboSnap and DROID-Sim show that Gaussian-splat environments can support trajectory replay, synthetic data generation, and policy evaluation at dataset scale [8].

This landscape rules out a weak framing: WorldEpisode is not the first format that connects robot data and Gaussian-splat worlds. That space is already occupied by asset standards and method papers. The remaining opening is the contract connecting all of them: the persistent, auditable binding between a robot-learning episode and the versioned world in which it occurred.

WorldEpisode targets that missing semantic layer. The contribution is a formal semantic contract, not a wrapper around existing libraries. It is:

- **storage-neutral**: serializable through LeRobotDataset v3, Rerun, NCore, MCAP, or a reference package;
- **representation-neutral**: compatible with splats, meshes, NeRFs, point clouds, and future assets;
- **runtime-neutral**: transferable to Isaac Sim, MuJoCo, SAPIEN, Genesis, and other simulators;
- **loss-explicit**: conversions may be lossy, but never silently lossy.

The paper’s core claim is therefore not a new monolithic file format. It is an executable interchange profile for world–episode semantics: identity, frames, clocks, action contracts, representation roles, events, provenance, uncertainty, and lineage-safe evaluation splits. The current artifact includes a JSON Schema, semantic validator, round-trip binding artifacts, an active public LeRobotDataset v3 conversion experiment, hand-authored independent fixtures, and controlled experiments for replay drift, leakage, and counterfactual augmentation.

2 Problem Statement and Scope

WorldEpisode binds an episode to a world revision. The base state is:

$$W_e(t) = W^{(r)} \oplus \Delta W_e(t), \quad (1)$$

where $W^{(r)}$ is an immutable, content-addressed world revision, $\Delta W_e(t)$ is the ordered sequence of episode-specific state changes, e is a robot-learning episode, and t is expressed in an explicitly declared clock domain.

An episode is modeled as:

$$E = \langle M, W^{(r)}, B, K, F, C, O, A, X, V, P, Q \rangle, \quad (2)$$

where M is the manifest, B the embodiment, K task/outcome semantics, F the frame graph, C clocks, O persistent entities and representations, A action-space contract, X synchronized trace, V world deltas and events, P provenance, and Q quality and uncertainty records.

Equivalently, each episode induces a coupled graph object:

$$G_E = (G_I, G_{FC}, G_R, G_T, G_P), \quad (3)$$

where G_I is persistent entity identity, G_{FC} is the frame/clock graph, G_R maps entities to representation roles and assets, G_T records temporal state and interaction events, and G_P records provenance and derivation. A package is valid only if every edge crossing these graphs resolves to a declared node and every lossy projection into a binding emits a loss report:

$$\text{valid}(E) \Leftrightarrow \text{closed}(G_E) \wedge \text{hashed}(W^{(r)}) \wedge \forall b \in B, \exists L_b. \quad (4)$$

Here $\text{closed}(G_E)$ means that entity, frame, clock, action, asset, and event references are total over the declared graphs; $\text{hashed}(W^{(r)})$ means the base world revision is content-addressed; and L_b is a machine-readable report of preserved, externalized, approximated, and discarded semantics for binding b .

Version 1 is intentionally narrow: rigid tabletop manipulation with fixed-base single- or dual-arm robots. Humanoids, locomotion, deformables, fluids, and multi-agent worlds are later profiles. This keeps conformance executable rather than aspirational.

WorldEpisode reuses existing standards rather than replacing them. USD [1] carries composed simulation worlds, glTF carries Gaussian appearance assets [9], NCore carries capture and pose-graph conventions [6], Rerun carries physical-data recordings with column chunks [7], and ROS/MCAP carries logs. WorldEpisode specifies the semantic relationships those bindings must preserve.

3 The Five-Graph Model

3.1 Identity Graph

Every physical or logical entity receives a persistent identifier: robot, link, gripper, sensor, rigid object, articulated object, fixture, support surface, region, human, light, or semantic group. The same `entity_id` must be used by segmentation masks, bounding boxes, Gaussian groups, render meshes, collision bodies, simulator actors, language references, contact events, object trajectories, grasp events, and attachment events.

3.2 Frame and Clock Graph

Every spatial value declares source frame, target frame, units, handedness, up axis, transform direction, quaternion convention, validity interval, calibration revision, and uncertainty where available. Every stream declares a clock domain, device timestamp, host-receive timestamp where available, exposure interval for cameras, effective command time for actions, synchronization mapping, and estimated offset/drift/error. NCore’s explicit pose-graph convention demonstrates the kind of rigor WorldEpisode requires across the whole episode [6].

3.3 Representation-Role Graph

One physical entity can have many representations: Gaussian splat appearance, textured render mesh, metric surface mesh, depth point cloud, collision proxy, mass/inertia record, semantic affordance, or learned embedding. Each representation declares its role and an asset descriptor: URI, media type, SHA-256 digest, optional mirrors, optional embedded payload metadata, coordinate frame, units, validity interval, derivation, uncertainty, and license. The URI may be relative, HTTP(S), Hugging Face, object storage, OCI, IPFS, or another registered resolver. Portability comes from

deterministic resolution and digest verification, not from requiring every asset to live in one folder. The role field is essential: a splat may be valid for appearance but invalid for collision.

3.4 Temporal State and Event Graph

Episodes do not store only poses. They record interaction events: contact begin/end, grasp establish/release, attachment create/remove, support, containment, articulation state changes, spawn/despawn, topology changes, simulator reset, intervention, subgoal completion, and failure detection. Each event references entities, time, confidence, source, and optionally evidence.

3.5 Provenance and Derivation Graph

Every derived asset identifies source episodes, source sensors and intervals, reconstruction algorithm, software version, model/checkpoint hash, configuration, calibration revision, manual edits, source and output hashes, creator, license, and quality metrics. WorldEpisode maps to existing provenance and dataset metadata vocabularies rather than inventing an incompatible publication vocabulary.

4 Action Semantics, Revisions, and Loss Reports

4.1 Complete Action Semantics

An action vector is not portable without its operational contract. Each action channel must declare actuator, control mode, parameterization, reference frame, units, absolute/delta/velocity semantics, limits, command rate, command timestamp, effective timestamp, latency model, interpolation, normalization, gripper semantics, and missing-value policy. A vector such as $[0.01, 0, 0, 0, 0, 0, 1]$ is otherwise ambiguous: it could be a tool-frame delta, base-frame delta, velocity command, normalized policy output, or delayed target increment.

4.2 Immutable World Revisions

Each episode references one base world revision plus ordered episode deltas. A new revision is required when world geometry, metric scale, calibration, collision proxies, physical parameters, entity decomposition, object identity, or task-relevant fixtures change. Content hashes allow two datasets to prove that they refer to the same world without relying on filenames.

4.3 Loss-Aware Conversion

WorldEpisode supports many bindings, but it does not promise impossible universal losslessness. Every converter emits a machine-readable report listing preserved, externalized, approximated, discarded, and warning fields. The standard permits lossy conversion; it forbids silent lossy conversion.

4.4 Implementation Artifact

We implement the contract as four executable components: a JSON Schema for structural validity, a machine-readable conformance requirement file, a semantic validator, and a round-trip experiment runner. The validator maps each diagnostic to a stable requirement identifier such as `TIME.001`, `FRAME.002`, or `ASSET.002`. The experiment runner takes a WorldEpisode package, exports binding-native artifacts plus sidecars, reimports them, compares a semantic projection, injects controlled

faults, and writes reproducible result artifacts. URDF Studio remains the initial implementation root, but the contract is represented independently in the public schema and validator.

The repository also includes an active LeRobotDataset v3 converter. It downloads bounded Parquet and metadata shards from the pinned public `lerobot/svla_so101_pickplace` SO-101 dataset, constructs a schema-valid WorldEpisode manifest and sidecar, exports a small LeRobot-Dataset v3 package, reads that package back, and asserts equality of source-native tensors and timestamp records. Source fields that LeRobot does not declare, such as camera extrinsics and controller latency, are not guessed; they are emitted in the conversion report as explicitly source-absent. The split-leakage tool similarly downloads bounded ArmnetBench LeRobot shards, writes `world_lineage` and split-manifest artifacts, and trains an executable BC baseline over the real LeRobot tensors. The control-replay tool reads the exported LeRobot trajectory, estimates the effective command delay from action/state alignment, writes the WorldEpisode action contract, executes the tested MuJoCo adapter, and emits an Isaac adapter mapping that is ready for an Isaac environment but explicitly marked untested in the committed report.

5 Controlled Experiments

We evaluate the contract with deterministic controlled experiments in the public repository. The suite isolates failure modes that larger robot experiments often hide: conversion loss, invalid physical semantics, timing drift, world-frame augmentation, and lineage leakage.

RQ1: semantic preservation across bindings. We implement executable export/import artifacts for seven bindings. Each binding writes a native payload and a WorldEpisode sidecar, then the experiment reimports both and checks exact equality against a 23-field semantic projection. Table 1 shows that common containers preserve only 17–39% of the contract natively, while the sidecar recovers all fields for dataset/log/world bindings. A Gaussian asset binding alone preserves only 17% natively and 30% with limited sidecar metadata, confirming that an asset format is not an episode–world contract. The LeRobot binding targets the current v3 storage layout rather than the initial dataset format described when the LeRobot library paper was submitted [3, 4].

Table 1: Semantic field retention in the pilot binding model.

Binding	Native	With WorldEpisode
LeRobot v3	0.261	1.000
Rerun RRD	0.348	1.000
NCore	0.304	1.000
MCAP/ROS2	0.261	1.000
OpenUSD/SimReady	0.391	1.000
glTF Gaussian asset	0.174	0.304

Active public LeRobot round-trip. To test an actual LeRobotDataset rather than only a capability model, we implement `LeRobotDataset v3` $\rightarrow$ `WorldEpisode` $\rightarrow$ `LeRobotDataset v3` on episode 0 of the pinned public SO-101 dataset `lerobot/svla_so101_pickplace` [5]. The converter downloads the real Parquet metadata and data shards, builds a schema-valid WorldEpisode manifest and sidecar, exports a compact LeRobotDataset v3 package, reads it back, and asserts exact equality. The episode contains 303 rows of six-dimensional action tensors, 303 state rows, 303 sample

timestamps, and two camera video timestamp ranges. The measured maximum absolute error is 0.0 for actions, states, sample timestamps, and video timestamp ranges. Three physical-frame records are preserved through the sidecar. Four source-absent semantics are explicitly reported rather than guessed: camera extrinsics, robot/world calibration, action units, and controller latency. No fields are discarded.

RQ2: physical-coherence fault detection. We inject 14 faults into the baseline package, including missing clock domains, incomplete cross-clock mappings, unknown frames, missing transform intervals, unknown event entities, missing representation roles, digest errors, action-contract omissions, non-content-addressed worlds, missing delta sequences, missing provenance, and missing quality records. The validator detects all 14 expected requirement failures with 1.000 recall and 0.933 precision. We also evaluate two hand-authored independent fixtures, both of which trigger the expected diagnostics. The single false-positive requirement is useful rather than harmful: removing an action reference frame triggers both `ACTION.001` and the downstream `FRAME.001` reference check.

Additional LeRobot unit sanity check. We include an offline mirrored first frame from `lerobot/svla_so101_pickplace`. Its six joint values have maximum magnitude 98.924 in degrees and 1.727 after radian conversion; treating degrees as radians would therefore violate the action/state unit contract. The corresponding SO-101 end-effector position $[0.1044, -0.00242, 0.07292]$ m is within a 0.5 m workspace bound. From the same mirrored frame we materialize a 10-frame LeRobot-style episode artifact with nine actions, command timestamps, effective timestamps, and a declared 20 ms latency model. This artifact is used as the recorded-episode fixture for timing and action-contract checks.

RQ3: LeRobot control-loop replay. We ground the action contract in the LeRobot control loop, where policy inference may run asynchronously from a fixed-rate low-level controller and commands may be queued, chunked, interpolated, or consumed later [3]. On the public SO-101 LeRobot trajectory used above, WorldEpisode records policy-output, enqueue, queue-consume, and motor-receive timestamp semantics, plus zero-order-hold interpolation and a constant-frame latency model. Estimating the effective delay on the calibration prefix gives four 30 Hz frames, or 133 ms. On the validation suffix, naive command-time alignment has joint RMSE 4.732 degrees, while timestamp-aware alignment has 1.862 degrees, a $2.54\times$ improvement. In a tested MuJoCo six-joint position-servo replay, naive command-time replay has RMSE 3.425 degrees and timestamp-aware replay has 1.563 degrees, a $2.19\times$ improvement. The Isaac adapter is emitted as a ready mapping from the same contract, but is not tested in this environment.

RQ4: synthetic counterfactual robustness. We evaluate a shifted-camera/object-displacement regression task. Observations-only nearest-neighbor control succeeds on 0.292 of shifted test cases. Adding unstructured 3D side information increases success to 0.342. WorldEpisode-style world-frame identity and counterfactual camera augmentation increase success to 0.592 over 120 deterministic test cases.

RQ5: lineage-safe splits. We also run a public LeRobot split audit on ArmnetBench v0.1’s single-arm SO-101 LeRobot release [2]. From the dataset metadata we derive eight WorldEpisode-style `world_lineage` hashes, one for each task-scene/camera-layout group. We train the same Torch MLP state-action behavioral-cloning baseline on two splits of the 400 teleoperated reference demonstrations. A random episode split trains on 320 episodes, tests on 80, and leaks all test scene

lineages into training. It obtains 0.850 offline BC imitation success under a fixed normalized-RMSE tolerance of 0.25, with mean episode normalized RMSE 0.183. The scene-disjoint split holds out the two eye-drop scene lineages, trains on 300 episodes, tests on 100, and has zero lineage leakage. Its offline BC success drops to 0.000 and mean episode normalized RMSE rises to 0.363, a $1.98\times$ increase. This is an offline action-imitation benchmark, not a physical rollout claim; the split manifest is directly reusable for native LeRobot ACT or Diffusion Policy training.

We generate episodes with world/entity lineage labels and evaluate a memorization policy. A random episode split leaks every tested world/entity pair and obtains 1.000 accuracy. World-disjoint and entity-disjoint splits have zero measured leakage and reduce memorization accuracy to 0.500 and 0.562 respectively. This pilot demonstrates why split manifests need lineage constraints.

6 Discussion

Not another monolithic binary. WorldEpisode has a small semantic core and multiple bindings. LeRobotDataset v3 is the policy-training view; Rerun is the multirate physical-data view; NCore is the sensor/reconstruction capture view; MCAP/ROS is the raw log view; OpenUSD/SimReady is the composed simulation-world view; glTF or SPZ is the Gaussian asset view; and a reference directory is the conformance profile.

Lineage-safe evaluation. Episode-level random splits can leak the same reconstructed room, physical object, Gaussian asset, source video, world revision, or generated asset lineage across training and evaluation. WorldEpisode supports split constraints such as `world_lineage`, `physical_entity`, `source_capture`, `reconstruction_run`, and `generated_asset_family`.

Standards alignment. The project aligns with standards bodies rather than antagonizing them. Static 3D robot-map exchange, humanoid dataset lifecycle, OpenLABEL annotation, glTF Gaussian splats, USD composition, and NCore capture conventions are all useful neighbors. WorldEpisode is an executable profile and bridge across those layers.

Scope. The controlled experiments are deliberately scoped to expose individual failure mechanisms rather than to claim full robot-learning generality. They do not replace a large multi-lab benchmark, but they do establish executable tests for the core methodological risks: conversion loss, invalid physical semantics, replay timing drift, and lineage leakage. The next empirical step is to run the same protocol on larger multi-camera manipulation datasets and independently maintained adapters.

7 Conclusion

WorldEpisode reframes the opportunity away from “LeRobot plus Gaussian splats in one file” and toward persistent, verifiable world-episode interoperability. Gaussian splats remain a high-value profile, but the durable contribution is a storage-neutral contract for immutable world revisions, persistent entity identity, frame and clock graphs, action semantics, representation roles, events, provenance, uncertainty, loss-aware conversion, and lineage-safe evaluation. The implementation turns those requirements into executable checks and reports measurable effects for conversion loss, fault detection, replay timing, counterfactual augmentation, and leakage-safe splitting. The next step is to scale the same protocol to larger multi-robot datasets and independently maintained bindings.

References

- [1] Alliance for OpenUSD. OpenUSD: Universal scene description. <https://openusd.org/>, 2026. Accessed 2026-07-13.
- [2] Armnet. ArmnetBench v0.1: LeRobot single-arm SO-101 release. https://huggingface.co/datasets/armnet/armnetbench_v01_lerobot_so101, 2026. Pinned revision 2e5e89aee0e7f081078d9d6ab3b279fc4b83ea84; accessed 2026-07-13.
- [3] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, Mustafa Shukor, Jess Moss, Alexander Soare, Dana Aubakirova, Quentin Lhoest, Quentin Gallouedec, and Thomas Wolf. LeRobot: An open-source library for end-to-end robot learning. In *International Conference on Learning Representations*, 2026. arXiv:2602.22818.
- [4] Hugging Face. LeRobotDataset v3.0. <https://huggingface.co/docs/lerobot/lerobot-dataset-v3>, 2026. Accessed 2026-07-13.
- [5] LeRobot. svla_so101_pickplace: A public LeRobotDataset v3 dataset. https://huggingface.co/datasets/lerobot/svla_so101_pickplace, 2025. Pinned revision f641879e22172be7e8161d5e6c1503c2d2feb657; accessed 2026-07-13.
- [6] NVIDIA. NCore Specification: Data Conventions. <https://nvidia.github.io/ncore/data/conventions>, 2026. Accessed 2026-07-13.
- [7] Rerun. A new data layer for robot learning. <https://rerun.io/blog/data-layer-for-robot-learning>, 2026. Accessed 2026-07-13.
- [8] RoboSnap Authors. RoboSnap: One-Shot Real-to-Sim Scene Generation for Generalizable Robot Learning and Evaluation. <https://arxiv.org/html/2607.06699v1>, 2026. Accessed 2026-07-13.
- [9] The Khronos Group. KHR_gaussian_splatting. https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md, 2026. Accessed 2026-07-13.
- [10] The Khronos Group. Khronos Announces glTF Gaussian Splatting Extension. <https://www.khronos.org/news/press/glTF-gaussian-splatting-press-release>, 2026. Accessed 2026-07-13.